

Design Optimization of Computing Systems

Assignment 2: NOVA-KV, A Lock-Free, RCU-Based In-Memory Key-Value Store for HFT/ML-Serving Workloads

Deadline: 11:59 PM, 25th October, 2026

Maximum Marks: 200 (+20 bonus)

Team

Size: 2-4

NOVA-KV ("Nanosecond-Optimized, Versioned Architecture") is the name of the system you will build in this assignment: a single-node, in-memory key-value store designed for a high-frequency trading (HFT) desk. It doesn't exist yet. You will be writing it from scratch.

Here is the scenario it's designed for. In HFT, the "hot path" is the code that runs on every single incoming market update, the part that has to finish in microseconds because a slow answer means a missed trade. NOVA-KV is meant to sit on that hot path. It holds the features an ML model checks before every decision, and it has to answer instantly, every single time.

The one thing NOVA-KV must never do is make a reader wait. A reader that gets stuck, because a writer is holding a lock, or because the whole system has frozen everything else just to clean up old data (a "stop-the-world pause"), is a reader that answers too late to be useful. So your job is to design a store where reads never wait on anything at all. One writer thread keeps publishing new, complete versions of the data, and readers always have a full, consistent version ready to read from immediately.

You'll build this using a technique called Read-Copy-Update (RCU). The idea is simple. The writer builds each new version off to the side, without touching what readers can currently see, and only at the very end switches everyone over to it in one instant (atomic) step. That means a reader is always looking at one complete snapshot of the data, never a version that's half old and half new, and never has to wait for the writer to finish anything.

Targets:

What the system needs to do:

- Run as a single process on one machine, your own multicore laptop or a lab machine. No distributed systems work here, and no real networking beyond a simple benchmark harness talking to it in-process.
- Sustain 100,000+ read requests per second, purely from memory, on an 8-core machine.
- Handle exactly one writer thread. You do not need to support multiple writers running at the same time. That one writer applies its own updates in order before publishing them.

What the system does not need to do, at least not yet:

- Survive a crash. If the process dies, all the data is gone. This is a deliberate simplification for Parts A through D, not an oversight, so say so plainly in your write-up. Part E brings this back as an optional bonus.
- Guarantee full serializability. Something weaker is fine, and expected. What matters is that you state precisely which isolation level your design actually gives, instead of just saying it's "isolated."

This assignment is supposed to give you an idea regarding:

- how RCU (pointer-swap, epoch-based reclamation, hazard pointers) gives lock-free reads without sacrificing correctness. This is the same technique used in the Linux kernel and in DPDK (a framework for writing very fast networking code that also avoids locking on its hot path), and it's taken to production extremes by single-writer systems like TigerBeetle (see Part E).

- how to choose a storage layout (row-major/NSM vs. column-major/DSM vs. hybrid/PAX) for a specific workload instead of defaulting to whichever is idiomatic.
- how to bound worst-case pause time for memory reclamation instead of using an unbounded stop-the-world sweep.
- how to get atomicity and isolation for multi-key writes without a general-purpose transaction manager.
- how to benchmark a concurrent system: throughput at increasing core counts, tail latency, and the throughput/pause-time trade-off of a tunable parameter.

The assignment is presented in 4 parts plus one optional bonus part, all of which build on each other and use the same store and the same workload generator throughout:

- **Part A:** you build a working KV store, run it under three concurrency strategies (single-threaded, global-lock, reader-writer-lock), profile them on 8 cores, and pick a storage layout (NSM, DSM, or hybrid/PAX) backed by data.
- **Part B:** you replace the baseline's locking with RCU, giving lock-free snapshot reads through atomic pointer-swap, and add a bounded-pause reclamation strategy (epoch-based or hazard-pointer) for retired versions, with one internal setting exposed as a tunable parameter.
- **Part C:** you extend the store with atomic, isolated multi-key writes and a secondary index used for an index-nested-loop join, both built on the RCU mechanism from Part B.
- **Part D:** you benchmark Parts A, B, and C together: the baseline lock strategies, RCU with your tunable reclamation parameter, and the join/transaction path, and show whether the 100k+ sustained read RPS target is met.
- **Part E (bonus):** you add a write-ahead log so committed writes survive a process crash, without slowing down the lock-free read path.

Materials provided with this assignment: `workload_gen.h` and `workload_gen.cpp` (Appendix A), a shared workload library used unchanged across Parts A, C, and D. There is no starter store: you write `kvstore.cpp` / `kvstore.h` from scratch, implementing exactly the API in Appendix A. Do not pull in an existing concurrent hash-map library: building and reasoning about this layer is the point of the assignment. Build everything with `-std=c++17` or later.

Part A: Baseline and Storage Layout (50 marks)

Build a working KV store, run it under three concurrency strategies, then use those runs plus a short write-up to decide how the store's rows should be laid out.

- Implement the core store in `kvstore.cpp` / `kvstore.h`, exposing exactly the `KVStore` interface from Appendix A (`put`, `get`, `erase`). Internally it's a hash table (open addressing or chaining, your choice). Write a small sanity test (`kvstore_test.cpp`) that checks basic insert, lookup, overwrite, and erase before you move on.
- Write 3 driver programs, `single_thread.cpp`, `global_lock.cpp`, and `rwlock.cpp`, that run the workload generator against your store under (i) no concurrency, (ii) a single `std::mutex` held for every put/get, and (iii) a `std::shared_mutex` that lets readers run concurrently (`shared_lock`) but still serializes against the writer (`unique_lock`).
- Use the same underlying data structure across all three drivers. Only the synchronization should differ. This keeps the three drivers a fair baseline for Part B.
- Profile `global_lock.cpp` and `rwlock.cpp` at 1, 2, 4, and 8 reader threads (one background writer throughout), using `perf`, a profiling tool that ships with Linux (run `perf --version` to check it's there. Install with `sudo apt install linux-tools-generic` if

not). You'll use two of its subcommands:

- **perf stat -- ./<program>** runs your program and prints hardware counters for the whole run: total time, cache misses, instructions per cycle. Good for a quick before/after number. Run this against both `global_lock` and `rwlock`.
 - **perf record -- ./<program>** followed by **perf script** samples *where in your code* the CPU is spending time, thousands of times a second, and dumps that as a stack-trace log. On its own this log is unreadable: hundreds of thousands of lines. A **flame graph** turns it into a picture: one box per function, stacked to show the call chain, with the *width* of each box showing how much of the total time was spent there (not the height, height is just call depth). Use Brendan Gregg's FlameGraph scripts (`stackcollapse-perf.pl` then `flamegraph.pl`) to generate one from **perf script**'s output. Do this for both `global_lock` and `rwlock`, at 1, 2, 4, and 8 reader threads each.
 - Use the flame graphs to identify where the 8-core runs lose time. Two usual suspects:
 - **Lock acquisition**: threads spending visible time inside the lock's own acquire function itself (`lock()` or `lock_shared()`), i.e. actually waiting for the lock to be free.
 - **Cache-line bouncing**: a subtler cost that won't show up as "waiting" but still slows everything down. Every core has its own cache, but caches move data around in fixed-size chunks called cache lines (64 bytes on most CPUs). The lock variable itself lives in one such chunk. Every time any core acquires or releases the lock, it has to modify that chunk, which forces every *other* core that has it cached to throw away its copy and re-fetch the current one, even though nothing they actually care about changed. With 8 cores all hammering the same lock, that chunk keeps getting invalidated and re-fetched across cores ("bouncing" between caches), and this back-and-forth can cost more than the time spent doing real work, even when no thread is technically stuck waiting.
- Don't just guess which of these dominates. The flame graph and **perf stat**'s cache-miss counters should show you directly (e.g. time inside the lock's internals, or a high cache-miss rate that scales with thread count). Point to that evidence instead of asserting "it's the lock."
- Separately, write a short design note (≤ 3 of your 5 pages) on how to lay out each record in memory, arguing for one of three options:
 - **Row-major (NSM)**: store each record's fields contiguously, one full row after another (like a normal C struct in an array).
 - **Column-major (DSM)**: store each field in its own separate array, so every trade's price sits next to every other trade's price, and similarly for every other field.
 - **Hybrid (PAX)**: group records into fixed-size pages, and within each page, lay fields out column-major.

Argue for the one you think fits *this* workload (point gets that read a whole row at once, plus the multi-field join you'll build in Part C). Don't just default to row-major because it's the familiar choice. Back your argument with at least one **microbenchmark**: a small, focused test program you write that actually measures point-get latency (or throughput) under two of these layouts side by side, so your claim is backed by a real number you measured, not just an argument on paper. Asserting NSM is right without running this comparison won't earn these marks, even if NSM turns out to be the correct call.

Deliverable: `kvstore.cpp` / `kvstore.h`, `kvstore_test.cpp`, the three driver files, your perf outputs and flame-graph images, and a report of no more than 5 pages covering what your profiling showed about where the lock-based versions lose time, and your storage-layout decision.

Part B: RCU Reads and Bounded-Pause Reclamation (40 marks)

Part A shows the cost of locking readers against a single writer. In Part B, remove that cost entirely. The change must live inside the `KVStore` class itself: don't fork the store into a second implementation.

- Replace the global-lock/rwlock synchronization with RCU: the writer builds a new version of the table (or just the buckets it touched, your choice, as long as it's correct) off to the side, then publishes it with one atomic pointer-swap of a "current version" pointer. A reader takes exactly one atomic load of that pointer at the start of its operation, and sees a consistent snapshot for the rest of that operation no matter what the writer does afterward.
- You may assume a single writer thread for the whole assignment (no writer-writer concurrency to arbitrate).
- Old versions can't be freed the instant they're unpublished, since a reader may still be using one. Implement a bounded-pause reclamation strategy: either epoch-based reclamation (readers announce an epoch, and a version is freed once no reader is still in an older epoch) or hazard pointers (readers publish the pointer they're using, and a version is freed once no hazard pointer references it). Either works. State which one you picked and why.
- Add a private member controlling how many retired versions a reclamation pass frees at once, plus a public setter, `void set_reclaim_batch(int n)`, declared alongside `put/get/erase` in `kvstore.h` and defined in `kvstore.cpp`.
- The reclamation pass itself must be bounded: each call may free at most as many retired versions as the current `reclaim_batch` setting, and must never block a reader or the writer. A stop-the-world sweep over the entire retire list does not satisfy this.

A single `get()` or `erase()` call is a complete operation on its own: it acquires a snapshot internally and releases it when the call returns, so by itself this gives you a consistent view only for the duration of one call. That is enough for a point lookup, but it is not enough for a multi-step operation like the join you will build in Part C.2, which needs one snapshot held across many separate lookups. Appendix A.1 adds an explicit, reusable snapshot handle for exactly that case.

Deliverable: your modified `kvstore.cpp` / `kvstore.h` and the diff against your Part A version, a `README.md` describing your reclamation design and the tunable parameter, and a stress test (concurrent readers hammering `get` while the writer hammers `put`) showing no reader ever sees a torn or partially-updated row.

Part C: Atomic Multi-Key Writes and an Index-Nested-Loop Join (60 marks)

Parts A and B give you a store where a single key is always read consistently. Part C asks two more questions: can a write touching several keys be all-or-nothing from a reader's point of view, and can a join over two tables run without ever seeing a torn mid-update state, without adding a second locking scheme on top of RCU?

C.1: Atomic, Isolated Multi-Key Writes (30 marks)

- Add a `Transaction` class (Appendix A) whose `put()` stages writes and whose `commit()` publishes them all as one new version with a single pointer-swap, built the same way as a

single-key write in Part B. If a `Transaction` is destroyed without `commit()` being called, discard its staged writes.

- Atomicity requirement: a concurrent reader must observe either none or all of a committed transaction's writes, never a partial subset.
- Isolation requirement: any single read (including every step of the join in C.2) must see one consistent, unchanging snapshot for its whole duration, even if the writer commits several transactions while it runs.
- Design note: state precisely which isolation level this gives you. It is snapshot isolation, not serializability, and the two are not the same thing. In particular, explain **write skew**: a situation where two transactions each read the same data, each independently make a change that is safe on its own, and commit at the same time, but the combination of both changes together violates a rule that neither transaction broke by itself (the classic textbook example is two doctors on call each independently going off duty because the other one is still listed as on call, leaving nobody on call at all). Snapshot isolation does not prevent this. True serializability would. Also state what you give up by leaving durability out of scope (what happens to an in-flight or just-committed transaction if the process dies right now).

C.2: Secondary Index and Index-Nested-Loop Join (30 marks)

- Build a secondary index (a hash index, or a small B+-tree, a search tree commonly used to keep keys in sorted order while staying shallow and cache-friendly, your choice) on one non-primary-key column of a second table you add to the store (see Appendix A for the two-table shape used by the benchmark).
- Acquire exactly one `Snapshot` handle (Appendix A.1) at the very start of the join, and pass that same handle to every `keys()`, `index_get()`, and `get()` call the join makes. This is what keeps the whole join pinned to one unchanging version, even if the writer publishes several new versions while it runs: it must never block the writer, and must never return rows from two different versions.
- Enumerate the outer (primary) table's rows with `keys()` (Appendix A.1), under that same snapshot, rather than reusing whatever key list the workload generator happened to produce while loading data. The join's outer loop iterates over what `keys()` returns.
- Implement an index-nested-loop join: for each key `keys()` returned, probe the secondary index on the inner table (via the snapshot-scoped `index_get()`) and, for each match, fetch the full inner row with the snapshot-scoped `get()` to emit it.
- You don't need to support an index on more than one column, or keep the index consistent under concurrent schema changes. Assume the indexed column is fixed for the assignment.

Deliverable: your modified source with the diff against the Part B version, and a short `README.md` / design note covering both the isolation-level argument from C.1 and the index/join design from C.2.

Part D: Benchmarking Everything Together (50 marks)

Parts A, B, and C gave you a baseline, a lock-free read path, and a join/transaction layer. Part D measures all of it together and shows whether the 100k+ RPS target is actually met, and what it cost to get there.

D.1: Point-Query Throughput and the 100k RPS Target (17 marks)

- Using the workload generator (Appendix A) with its default parameters, benchmark point-get RPS for all four configurations (`single_thread`, `global_lock`, `rwlock`, and your Part B RCU store) at 1, 2, 4, and 8 reader threads, with one background writer thread active throughout. Alongside RPS, report p50, p99, and p999 latency: the typical response time (p50, the median), and how bad your worst 1% (p99) and worst 0.1% (p999) of requests get. A system can have great average throughput and still be unusable if its tail latency, that worst 0.1%, is bad, which is exactly why HFT systems care about it.
- Report the reader-thread count at which each lock-based configuration stops scaling (i.e. adding more threads stops increasing throughput). Separately, confirm whether the RCU configuration sustains 100,000+ RPS at 8 reader threads. If it doesn't, report the number you actually achieved and explain the remaining bottleneck, backed by a flame graph.

D.2: Reclamation-Batch Tuning (17 marks)

- Run the same point-get benchmark at 4 or more different reclaim-batch settings (for example, 8, 64, 256, and 1024).
- For each value, report two numbers: (a) sustained read RPS, and (b) the longest single reader stall you observed. A latency histogram is enough to pull the second number out. Together, these two numbers show what the batch size actually trades off: a bigger batch should mean higher throughput but a longer worst-case stall.
- You need two reference points to make that trade-off visible. First, run the same benchmark with reclamation disabled entirely (old versions are never freed). Second, implement a naive stop-the-world reclaim just for this comparison: freeze all readers, walk the entire retire list, and free everything at once. Report both numbers for each reference point too.

D.3: Join and Multi-Key Write Benchmarking (16 marks)

- Report index-nested-loop join throughput and latency at 8 reader threads with the background writer active, and compare it against the same join run with the writer paused. The difference between the two tells you what the concurrent writer actually costs a running join.
- Report multi-key transaction commit throughput as a function of transaction size (1, 4, and 16 keys per transaction). Also report how much your bounded reclamation strategy reduces worst-case reader pause compared to the naive stop-the-world reclaim from D.2.
- Your RCU design works by copying data instead of locking it (this pattern is called copy-on-write, or COW). Explain a workload where that copying approach helps little or even hurts, for example a very large transaction that forces copying a large fraction of a table, and explain why.

Deliverable: results, explanations, and answers for D.1–D.3, plus the raw perf/latency-histogram data backing each of them.

Part E (Bonus, Optional): Adding Durability (+20 marks)

Everything above assumes the process never crashes. A real trading desk won't run anything that loses its state on a restart.

The gap between “fast in-memory prototype” and “a system someone would trust with real money” is almost entirely this layer, not the throughput number. TigerBeetle, a real financial transactions database built around exactly this single-writer, no-row-locks pattern, didn't stop

at raw single-core throughput. It added checksummed, corruption-detecting durability guarantees on top, and that's the layer that made it a fundable product rather than a fast demo. Its public design document is a good model both for the guarantees to aim for and for how to write them down.

- Add a write-ahead log (WAL): every committed transaction is appended to an on-disk log before (or as part of) its RCU publish, and a restarted process replays the log to reconstruct the last committed state.
- The WAL must not add per-read overhead. Reads stay exactly as lock-free as in Part B/C. Where you put the `fsync`, and how you batch writes together to amortize its cost, is your design decision to justify. This batching pattern, delaying a commit briefly so several transactions can share a single `fsync` instead of each paying for their own, is called **group commit**: it trades a small amount of added latency per transaction for a large reduction in total disk syncs, which is usually a good trade when transactions are arriving faster than individual `fsync` calls can keep up.

Deliverable: your WAL implementation and diff, a short note on your `fsync`/group-commit policy and its throughput cost (re-run D.1's write-side numbers with and without the WAL), and half a page on what's still missing before NOVA-KV could run on an actual desk.

Submission Guidelines

Zip all the files into a single zip file named:

`Asgn2_<RollNumber1>_<RollNumber2>[_<RollNumber3>][_<RollNumber4>].zip`

and upload it on moodle. Only one submission per team.

Evaluation Guidelines

Total marks for the core assignment = 200, distributed below. Part E is optional bonus credit on top.

Part	Item	Marks
Part A: Baseline & Storage Layout	Core KV store + 3 concurrency variants (single-thread / global-lock / rwlock)	20
	NSM vs. DSM vs. PAX write-up, backed by data	20
	Baseline profiling / flame-graph report	10
Part A Total		50
Part B: RCU Reads & Bounded Reclamation	RCU pointer-swap + lock-free snapshot reads	15
	Epoch- or hazard-pointer reclamation + tunable batch size	15
	README + no-torn-read correctness test	10
Part B Total		40
Part C: Atomicity/Isolation & Join	C.1: Multi-key atomic write path implementation	20
	C.1: Design note (isolation level, why not durable)	10

Part	Item	Marks
	C.2: Secondary index + index-nested-loop join implementation	21
	C.2: Design note	9
Part C Total		60
Part D: Benchmarking for 100k+ RPS	D.1: Point-query scaling, 1-8 cores, 100k+ RPS check	17
	D.2: Reclamation-batch tuning (throughput vs. pause bound)	17
	D.3: Join & multi-key write benchmarking	16
Part D Total		50
Grand Total		200
Part E (bonus, optional): Durability	WAL + replay, without slowing the RCU read path	+20

Appendix A: KV Store API and Workload Generator

This appendix fixes two things everyone builds against: the exact interface `kvstore.cpp` must implement at each part, and the workload generator that exercises it. `workload_gen.cpp` never touches your hash table directly. It only calls the API below, so it links unchanged against every configuration you build, and Part D's comparisons end up being between your implementations rather than between different test setups.

A.1 API Contract

Required from Part A onward, implemented in `kvstore.cpp`, declared in `kvstore.h`:

```
class KVStore {
public:
    KVStore();
    ~KVStore();
    KVStore(const KVStore&) = delete;
    KVStore& operator=(const KVStore&) = delete;
    // move constructor/assignment: left undeclared (see note below)

    bool put(long key, const void* value, size_t value_len);
    bool get(long key, void* value_out, size_t value_out_len) const;
    // returns true if found (value copied into value_out)
    bool erase(long key);
    // returns true if the key was present and is now removed
};
```

Added in Part B, alongside the API above:

```
// added to KVStore:
void set_reclaim_batch(int n);
// sets how many retired versions one reclamation pass may free
```

Added in Part C, alongside the API above:

```

class Transaction {
public:
    explicit Transaction(KVStore& store);
    ~Transaction(); // discards any uncommitted writes
    Transaction(const Transaction&) = delete;
    Transaction& operator=(const Transaction&) = delete;

    void put(long key, const void* value, size_t value_len);
    bool commit();
    // true if committed; all staged writes become visible in one publish
};

// added to KVStore:
bool create_index(size_t field_offset, size_t field_len);
// builds a secondary index over a byte range within each stored value

int index_get(const void* field_value, size_t field_len,
              long* keys_out, int max_keys) const;
// returns the number of matches (up to max_keys), primary keys in keys_out

```

A single `get()`, `erase()`, or (as just shown) `index_get()` call acquires a snapshot internally and releases it when the call returns, so each call on its own is only guaranteed to be consistent for its own duration. That's enough for a single point lookup, but the join in Part C.2 needs one snapshot held across many separate calls: one for enumerating the outer table, then one `index_get()` and one `get()` per matching row, all seeing the exact same version throughout. The API above has no way to do that, so Part C also adds an explicit, reusable snapshot handle, plus a way to enumerate a table's keys under it:

```

// added to KVStore:
class Snapshot {
public:
    Snapshot(Snapshot&&) noexcept;
    Snapshot& operator=(Snapshot&&) noexcept;
    ~Snapshot();
    Snapshot(const Snapshot&) = delete;
    Snapshot& operator=(const Snapshot&) = delete;
};

// An opaque, move-only handle on one published version. For as long as a
// Snapshot object is alive, the version it was acquired against must stay
// pinned by your Part B reclamation scheme (exactly as if a reader were
// still "inside" a call against that version), even across many separate
// get() / index_get() / keys() calls using that same handle.

Snapshot acquire_snapshot() const;
// hands back a Snapshot pinned to the current version, for the caller to
// reuse across multiple subsequent calls instead of letting each call take
// and release its own

bool get(const Snapshot& snap, long key,
         void* value_out, size_t value_out_len) const;
int index_get(const Snapshot& snap, const void* field_value, size_t field_len,
              long* keys_out, int max_keys) const;
size_t keys(const Snapshot& snap, long* keys_out, size_t max_keys) const;
// enumerates up to max_keys primary keys present in the table under snap;
// returns the number written to keys_out

```

The index-nested-loop join in C.2 should acquire exactly one `Snapshot` up front and pass it to every `keys()`, `index_get()`, and `get()` call it makes, rather than calling the single-argument

overloads (which each take their own, independent snapshot and would let the outer enumeration, the index probes, and the row fetches each see a different version).

`get()` copies the matched value out before returning, rather than handing back a pointer into your internal storage. The copy happens while the RCU read-side critical section from Part B (or, for the snapshot-scoped overload, the `Snapshot`'s pin) is still open, so the caller never holds a pointer that could be reclaimed out from under it. Keep the same copy-out convention for `index_get`'s matches.

Neither `KVStore` nor `Transaction` should be copyable or movable: both own state (a current-version pointer, staged writes) that a shallow copy would corrupt. Both classes declare a destructor, and a user-declared destructor already suppresses the compiler from generating a move constructor and move assignment operator, so those two can simply be left undeclared, as shown above. It does *not* suppress the copy constructor or copy assignment operator, though: those stay implicitly generated, and usable, unless deleted explicitly. That's why `KVStore(const KVStore&) = delete;` and `KVStore& operator=(const KVStore&) = delete;` (and the same two lines for `Transaction`) are written out explicitly above. Don't rely on the destructor alone to take care of copying; it doesn't.

A.2 Workload Generator

Provided as `workload_gen.h` (below) and `workload_gen.cpp`, attached separately. It creates a `Trade` table (the high-write-rate table point queries read against) and, when a join is requested, a small `Instrument` table indexed by symbol, standing in for joining a live tick against slow-changing instrument metadata. One writer thread loads $m \times n$ trades, as single puts or batched into `Transactions` of `txn_size` trades depending on `cfg.use_txn`. Then `num_readers` threads issue point gets, mostly against a hot subset and occasionally against a random key, for `duration_sec` seconds, each recording its own call latencies into a private vector so the recording itself never contends between reader threads. `workload_run()` joins every thread, merges the vectors, and reports throughput and latency percentiles.

```
// workload_gen.h
#pragma once
#include "kvstore.h"
#include <string>

struct WorkloadConfig {
    int m = 1000, n = 100; // m * n trades inserted by the writer
    int hot_every = 100;   // 1 in N trades is re-read repeatedly by readers
    int num_readers = 8;   // concurrent reader threads
    int duration_sec = 5;  // how long readers run once loading is done
    bool use_txn = false;  // false: put() per trade, true: batch into Transaction
    int txn_size = 1;      // trades per transaction, used only if use_txn
    bool use_join = false; // false: point gets only,
                          // true: also probe the instrument index
};

struct WorkloadResults {
    long total_ops = 0;
    double rps = 0;
    double p50_us = 0, p99_us = 0, p999_us = 0;
};

void workload_run(KVStore& trades, KVStore* instruments,
                 const WorkloadConfig& cfg, WorkloadResults& out);
void print_results(const std::string& label, const WorkloadResults& r);
```

`workload_gen.cpp` only calls the KVStore / Transaction API above, so you shouldn't need to modify it. Build everything, including your own `kvstore.cpp`, with `-std=c++17` or later.

A driver just supplies the store and a config. For example, Part A's `global_lock.cpp` would end with:

```
int main() {
    KVStore trades;
    WorkloadConfig cfg;
    cfg.num_readers = 8;
    cfg.duration_sec = 5;
    WorkloadResults res;
    workload_run(trades, nullptr, cfg, res);
    print_results("global_lock", res);
}
```